

Web Security – Web Attacks and Defences

IHC – Practical Approach to Cyber Security (PACS)

Modular Assignment 3

Student: Nitesh Vishwakarma

1. Introduction and Objective

This assignment studies common web security vulnerabilities in a controlled learning environment. The focus is on HTTP/HTTPS and the client-server model, SQL injection, reflected and stored cross-site scripting (XSS), cross-site request forgery (CSRF), and web tracking and privacy.

All demonstrations are intended for deliberately vulnerable applications or local test code such as OWASP Juice Shop. No real third-party website is targeted.

2. Web Fundamentals

A web application normally uses a client-server architecture. A browser acts as the client and sends HTTP requests to a web server or application server. The server processes the request and returns an HTTP response.

HTTP commonly carries methods such as GET, POST, PUT, PATCH and DELETE. Requests and responses contain headers and, when required, a message body.

HTTPS is HTTP protected by TLS. TLS provides encrypted communication and helps the client authenticate the server through digital certificates. HTTPS protects data while it travels between the client and server, but application-level authentication, authorization and input validation are still required.

3. SQL Injection

SQL injection occurs when untrusted input is incorrectly incorporated into a database query. An attacker may be able to alter the intended meaning of the query.

Controlled demonstration example:

Vulnerable pattern:

```
query = "SELECT * FROM users WHERE username = " + username + ""
```

If untrusted input is concatenated directly into SQL, the application may interpret part of the input as SQL syntax.

A safer pattern is a parameterized query:

```
query = "SELECT * FROM users WHERE username = ?"
```

```
cursor.execute(query, (username,))
```

The parameter is handled as data instead of being interpreted as part of the SQL command.

Mitigations include parameterized queries/prepared statements, input validation, least-privilege database accounts, safe error handling, and regular security testing.

4. Reflected XSS

Reflected XSS occurs when attacker-controlled input is returned by the server and inserted into a page without appropriate output encoding or sanitization.

Controlled example:

Vulnerable:

```
return "Search: " + search_term + ""
```

If the value is inserted as raw HTML, browser-executable content may be interpreted.

Safer approach:

```
return "Search: " + html_escape(search_term) + ""
```

The application should treat user input as data and encode it for its output context. Content Security Policy (CSP) can provide an additional layer of protection.

5. Stored XSS

Stored XSS occurs when malicious input is saved by the application and later displayed to other users. A common example is an unsafe comment or profile field.

Vulnerable pattern:

```
save_comment(comment)
```

Later:

```
render_raw(comment)
```

Safer design validates and sanitizes data where appropriate and, importantly, applies context-appropriate output encoding when displaying it. HTML sanitization is useful when an application intentionally allows a limited amount of HTML. A restrictive Content Security Policy can provide defense in depth.

Testing should be performed only with harmless test payloads in the authorized training application.

6. CSRF

Cross-Site Request Forgery occurs when a victim's browser is induced to send an unwanted authenticated request to a website where the victim is already logged in.

For example, a vulnerable state-changing endpoint might accept a request based only on the user's session cookie.

A common defense is a server-generated CSRF token. The application places the token in the legitimate form and verifies it when the request is received. Requests without the correct token are rejected.

Additional controls include SameSite cookie attributes, checking request origin where appropriate, requiring re-authentication for sensitive actions, and avoiding state changes through GET requests.

7. Web Tracking and Privacy

Web tracking can use mechanisms such as cookies, third-party resources, advertising pixels, browser storage, and other identifiers to observe activity across pages or services.

Privacy-enhancement measures include:

- Minimize collection of personal data.
- Avoid unnecessary third-party scripts.
- Use restrictive cookie settings and appropriate SameSite/Secure/HttpOnly attributes.
- Provide clear consent and privacy information where required.
- Reduce retention of tracking data.
- Use Content Security Policy and related browser security controls.
- Review third-party domains and permissions regularly.
- Provide users with meaningful privacy choices.

Privacy protection should be considered during application design rather than added only after deployment.

8. Attack and Defense Summary

SQL Injection — Risk: unauthorized database queries or data access. Defense: parameterized queries, validation and least-privilege database access.

Reflected XSS — Risk: attacker-controlled content executes in a victim's browser. Defense: context-aware output encoding, input handling and CSP.

Stored XSS — Risk: malicious stored content affects users who view it. Defense: safe storage/display practices, sanitization where HTML is allowed, output encoding and CSP.

CSRF — Risk: an authenticated victim performs an unwanted state-changing action. Defense: anti-CSRF tokens, SameSite cookies and appropriate origin checks.

Tracking/privacy — Risk: excessive collection or sharing of user activity. Defense: data minimization, consent, reduced third-party tracking, secure cookie settings and retention controls.

9. Testing Approach

The demonstrations should be performed in OWASP Juice Shop or another explicitly authorized local training environment.

Testing process:

1. Start the deliberately vulnerable application.
2. Identify the relevant input or request.
3. Observe the normal application behavior.
4. Use a harmless test value to demonstrate the vulnerability.
5. Record the observed behavior.
6. Apply the mitigation.
7. Repeat the test.
8. Confirm that the unsafe behavior is no longer possible.
9. Document the result.

The purpose is to understand the vulnerability and validate the defense, not to target an external service.

10. Observations and Learnings

The main observation is that many web vulnerabilities result from treating untrusted input as trusted data or from relying on browser behavior without server-side controls.

HTTPS protects communication in transit but does not automatically prevent SQL injection, XSS or CSRF. Input handling, output encoding, authentication, authorization, secure cookies and server-side validation are complementary controls.

Security controls should be applied at multiple layers so that failure of one control does not automatically expose the entire application.

11. Conclusion

This practical assignment demonstrates the basic operation and defenses of SQL injection, reflected XSS, stored XSS and CSRF, together with the fundamentals of HTTP/HTTPS and web tracking/privacy.

The recommended defenses focus on secure coding, parameterized database queries, context-aware output encoding, anti-CSRF protections, secure cookie configuration, data minimization and controlled third-party tracking. Testing in a deliberately vulnerable application provides a safe way to understand both the attack behavior and the effectiveness of the mitigation.

12. References

1. OWASP Juice Shop – deliberately insecure web application for security training.
2. OWASP Web Security Testing Guide.
3. SQL Injection Playground – assignment-provided training resource.
4. XSS Game by Google – assignment-provided training resource.
5. Web Privacy Measurement Dataset – assignment-provided resource.
6. Assignment brief: IHC – Practical Approach to Cyber Security, Modular Assignment 3.

Appendix A – Annotated Safe Code Examples

```
# MA3 - Safe Web Security Demonstrations
# These examples are for local/authorized training only.

# 1. SQL Injection
# Vulnerable pattern (do NOT use in production):
query = "SELECT * FROM users WHERE username = '" + username + "'"

# Safer parameterized pattern:
query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,))

# 2. Reflected XSS
# Vulnerable pattern:
return "<h2>Search: " + search_term + "</h2>"

# Safer pattern:
from html import escape
return "<h2>Search: " + escape(search_term) + "</h2>"

# 3. CSRF defense - simplified Flask-style example
from secrets import token_urlsafe

csrf_token = token_urlsafe(32)

# Store token in the user's server-side session.
session["csrf_token"] = csrf_token

# For a state-changing request, compare the submitted token:
submitted = request.form.get("csrf_token")
if submitted != session.get("csrf_token"):
    abort(403)

# Also use appropriate cookie settings:
# Secure=True, HttpOnly=True, SameSite="Lax" or stricter where suitable.
```